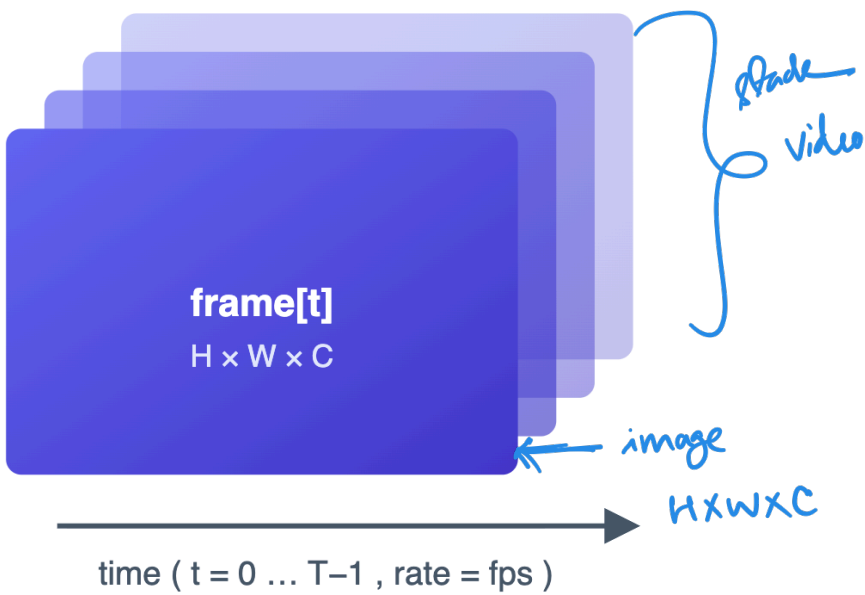


"T" each frames $\leftarrow$ (H x W x C) stacks of such images } a video
frames

A video is a 4-D tensor: $T \times H \times W \times C$

Decoded frames stacked along the time axis — the unit of computation for every model below.



Video AG =

Shape

T frames · H rows · W cols · C channels (RGB→3)

Memory (raw uint8)

bytes $\approx T \cdot H \cdot W \cdot C \rightarrow$ grows linearly with length

$\Rightarrow$ if you double video length $\Rightarrow$ double the no. of images
 $\Rightarrow$ doubling the memory

Codec & keyframes (GOP)

I = full keyframe · P/B = predicted from neighbors
seeking lands on the nearest I-frame, not any frame

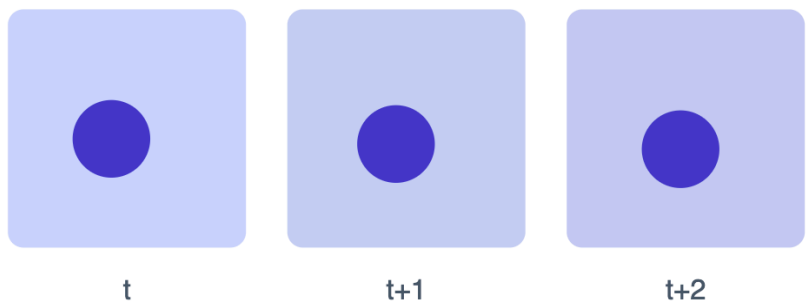


Image AG but on multiple frames

Two redundancies make raw video wasteful to model

1 · Temporal redundancy

Adjacent frames are almost identical — most pixels repeat.

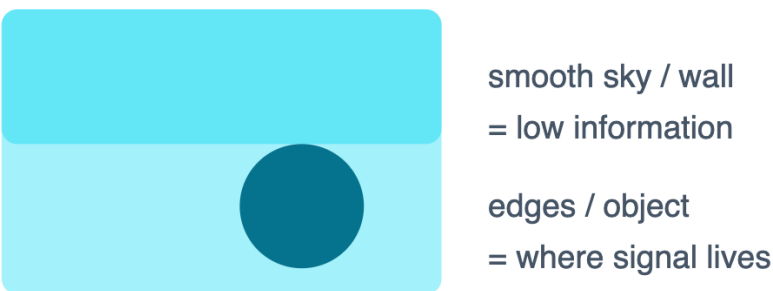


$\text{frame}[t+1] - \text{frame}[t] \approx 0$

Dense sampling wastes compute on near-duplicate frames → motivates frame sampling.

2 · Spatial redundancy

Within a frame, neighboring pixels are correlated.



Resize / patchify keeps structure

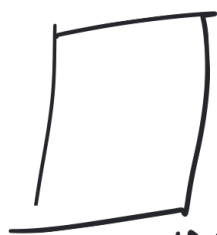
Drops resolution the model never needed → fewer tokens, same semantics.

Net effect: 10 min @ 30 fps = 18,000 frames — sample & resize before any model sees it.

Raw video

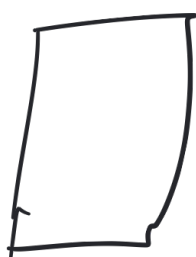
* adjacent frames are often almost the same

frame [1]



HxWxC

frame [2]

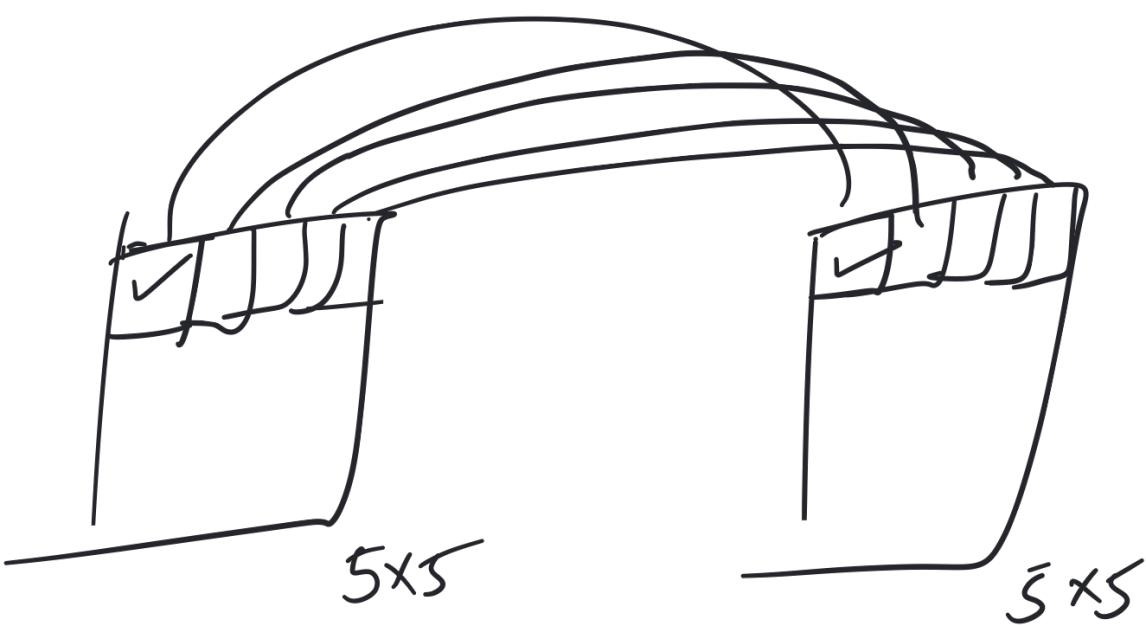


HxWxC

temporal redundancy
(detect it?)

* lot of repeated information, in betⁿ two frames

detect temporal redundancy $\Rightarrow$ find difference betⁿ two frames (MAE on pixels of the frames)



*Dedup

*many are duplicates

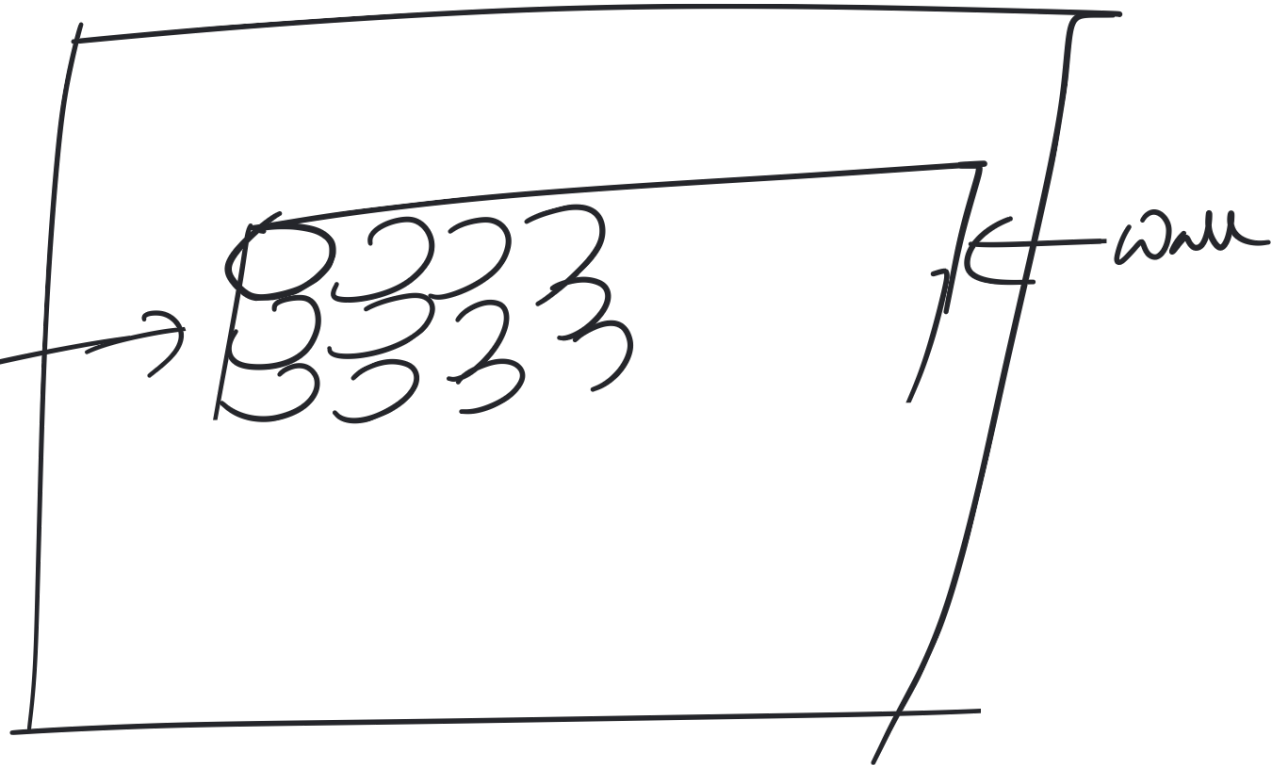
30 frame per second

~~30 fps~~ 30 fps x 10 minutes

18000 frame

Spatial Redundancy

pixels in the
nearby proximity
⇒ spatial redundancy



Raw video processing $\Rightarrow$ expensive

$\Rightarrow$ time - nearby frames repeat ✓ (MAE)

$\Rightarrow$ space - nearby pixels repeat ✓ (SSIM)

In one simple special case, further explained in the next section, the SSIM measure between x and y is:^[4]

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

with:

- μ_x the **pixel sample mean** of x ;
- μ_y the **pixel sample mean** of y ;
- σ_x^2 the **sample variance** of x ;
- σ_y^2 the **sample variance** of y ;
- σ_{xy} the **sample covariance** of x and y ;
- $c_1 = (k_1 L)^2$, $c_2 = (k_2 L)^2$ two variables to stabilize the division with weak denominator;
- L the **dynamic range** of the pixel-values (typically this is $2^{\text{\#bits per pixel}} - 1$);
- $k_1 = 0.01$ and $k_2 = 0.03$ by default.

$\text{SSIM} = 1 \Rightarrow$ frames are similar
 $\text{SSIM}(\text{2frames}) = 0 \Rightarrow$ frames are diff
 $\rightarrow$ Brightness sim
 $\rightarrow$ Contrast sim
 $\rightarrow$ Structural sim
(are edges/patterns arranged similarly)

... *Big change between two frames*

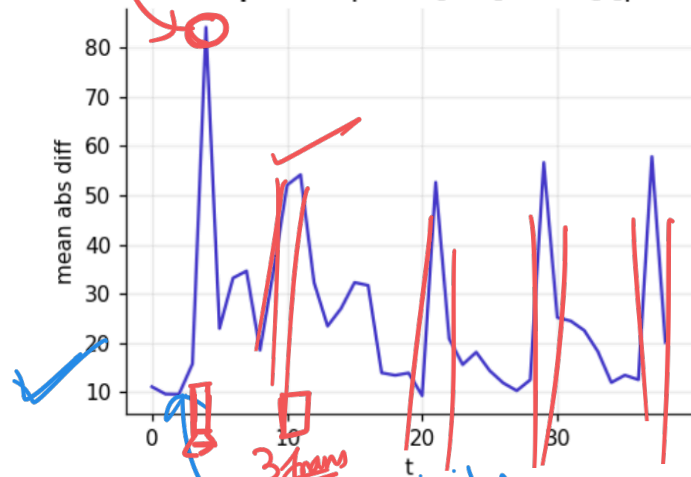
mean consecutive-frame MAE : 25.530 (0 = identical)

mean consecutive-frame SSIM: 0.4976 (1 = identical)

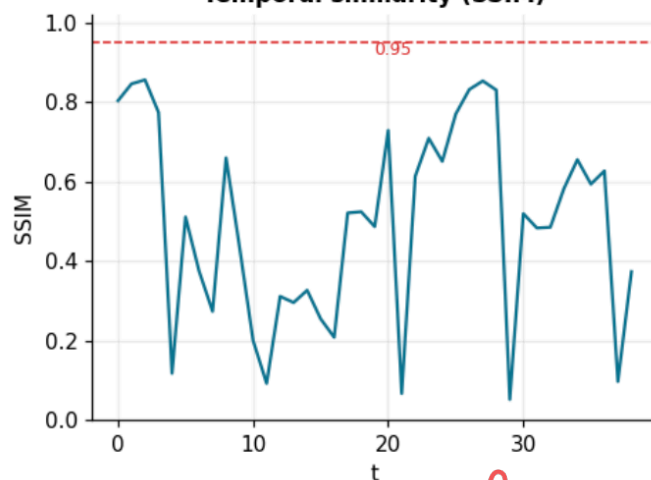
~31% of pixels are low-detail by this proxy (spatial redundancy)

(ideal) high SSIM

Temporal Δ : $|\text{frame}[t+1] - \text{frame}[t]|$

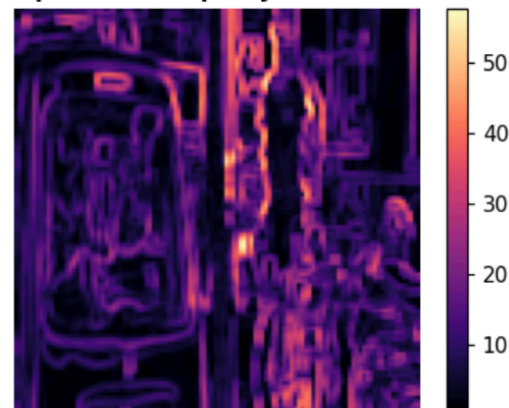


Temporal similarity (SSIM)



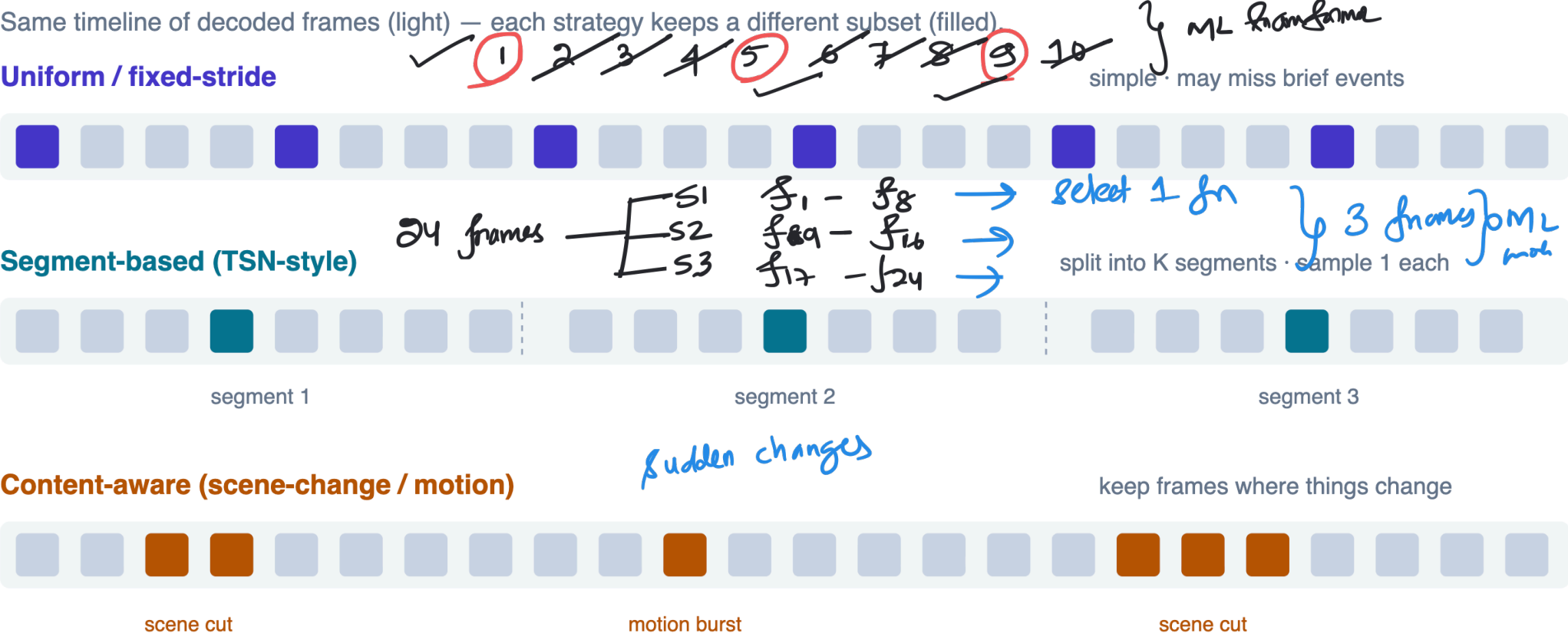
structural sim = less = 0.4

Spatial detail proxy (local std)



*31% of pixels are low detail
1/3 = less detailed
⇒ some spatial redundancy but not very high*

Frame sampling: which frames actually reach the model

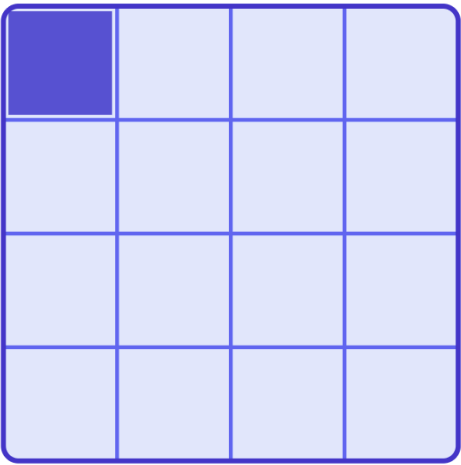


Trade-off: uniform is cheapest & reproducible; content-aware preserves rare events at higher preprocessing cost.

Tokenization: turning pixels into a Transformer sequence

A frame is cut into non-overlapping $P \times P$ patches $\rightarrow$ each patch is linearly projected to one token.

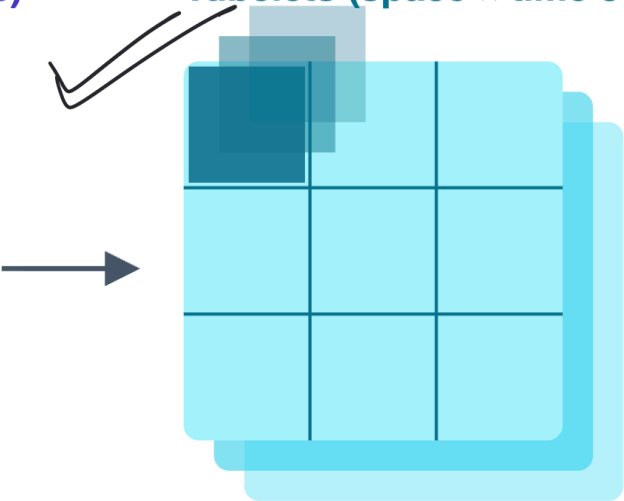
Per-frame patches (ViT-style)



$(H/P) \cdot (W/P)$ tokens per frame

$$224/16 = 14 \rightarrow 14 \cdot 14 = 196$$
$$\times 100 \text{ frames} = 19,600 \text{ tokens}$$

Tubelets (space $\times$ time cubes)



one token spans $P \times P \times t$ pixels

tubelet depth t collapses
the time axis $\rightarrow \div t$ fewer tokens

Why it matters

- Token count drives attention cost (quadratic in sequence length).
- Each token carries a 3-D position: **(x, y, t)**
 - unlike 1-D text positions.
- Bigger P or tubelet depth t
 $\rightarrow$ fewer tokens, coarser detail.
- [CLS] token aggregates the clip for classification.

16x16

frames
↓
8 × 160 × 160 × 3
↑
each frame size

total of k=2

16x16xk

1 × 160 × 16 × 3

1 × 160 × 160 × 3

$$\frac{160 \times 160 \times 3}{16 \times 16} = 10 \times 10 \times 3 \times 4$$

10 × 10 × 3 × 4

Brain

$$\textcircled{10} \times \frac{160 \times 160 \times 3}{1}$$

$$160 \times 160 \times 3$$

$$\hline 16 \times 16$$

=

$$\textcircled{10 \times 10 \times 10 \times 3}$$

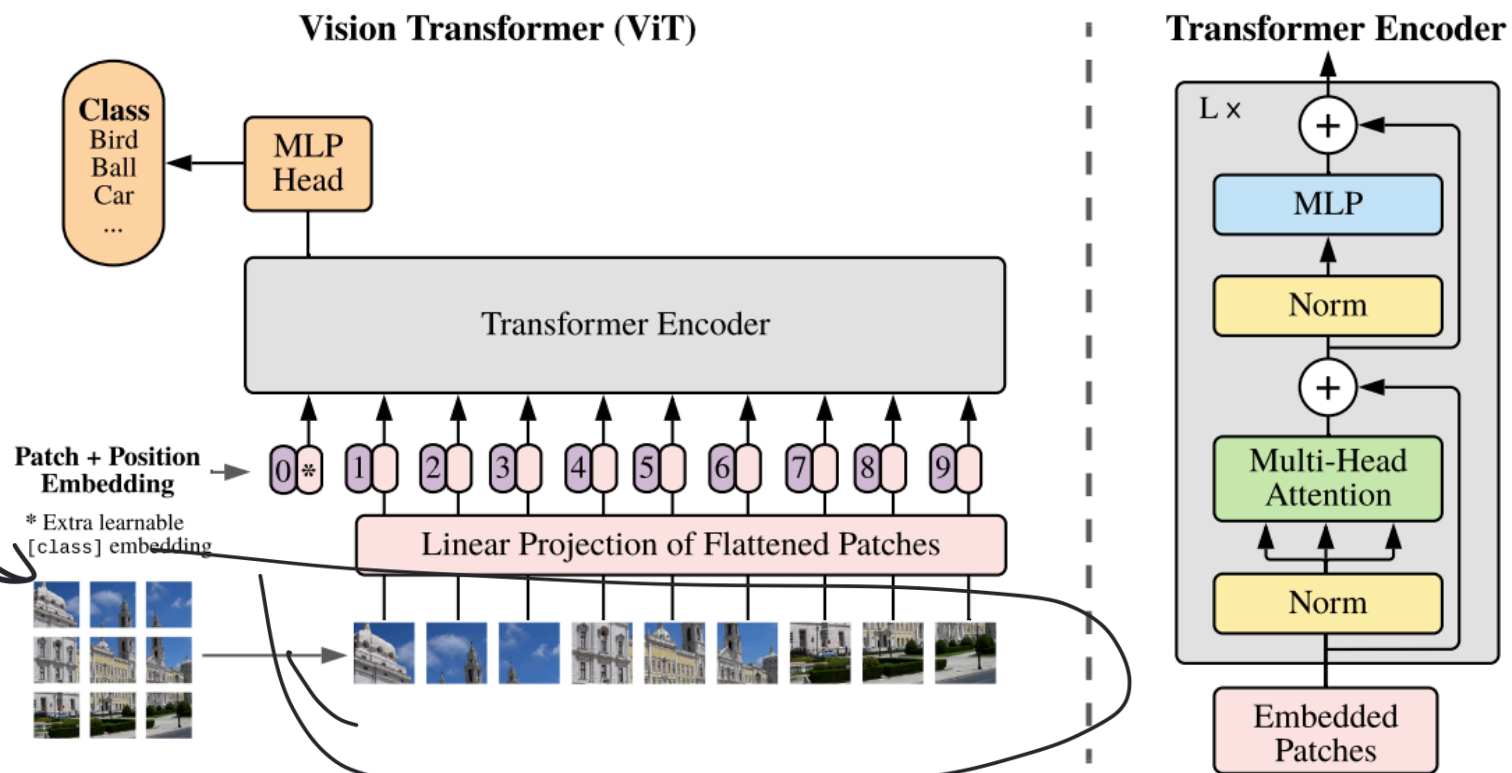
Patcha

3 such images

16 X 16 patches

each patch = a token

Position Embedding





Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

I am good
 ↓ ↓ ↓ ↓
 0 am oo d

$$10 \times (160 \times 160 \times 3)$$

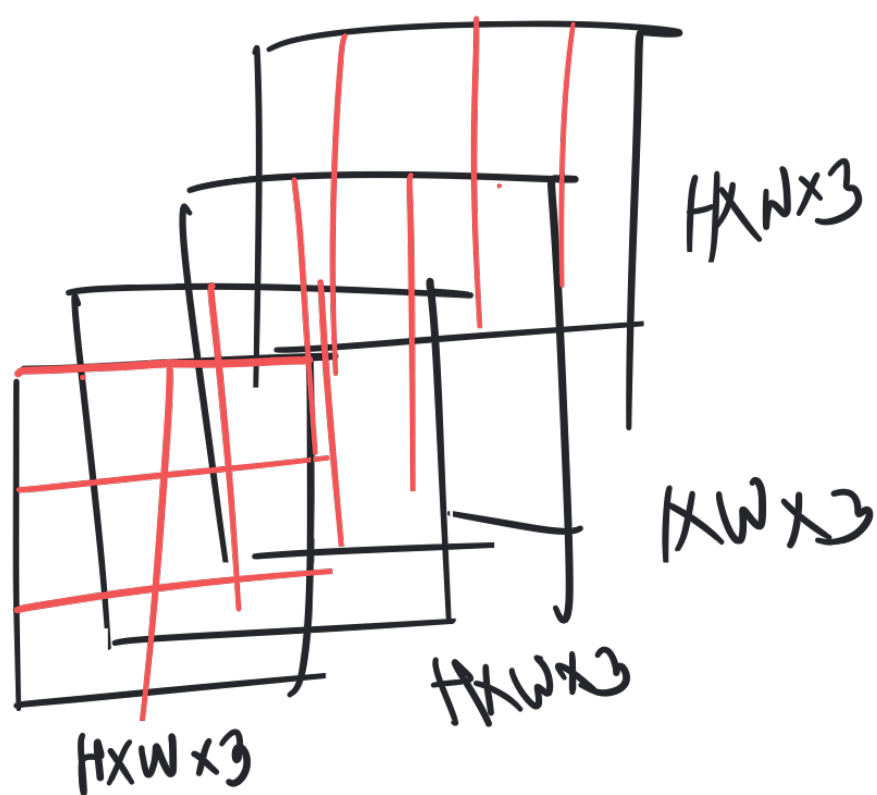
$$\text{Patch} = 16$$

$$\text{patch depth} = 2$$

$$\text{Each patch} = 2 \times 16 \times 16 \times 3$$

$$T, h, w, c = 10, 160, 160, 3$$

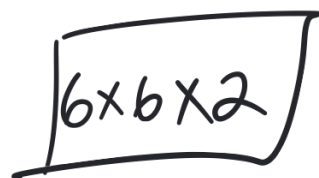
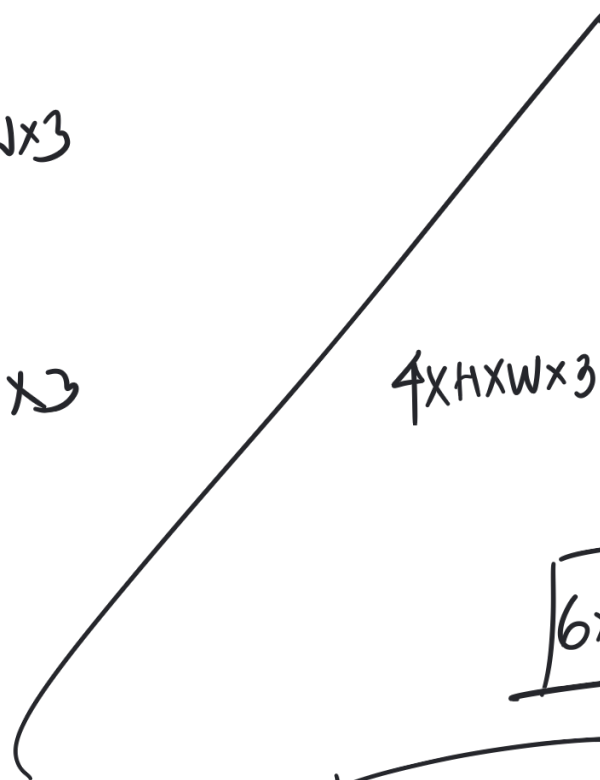
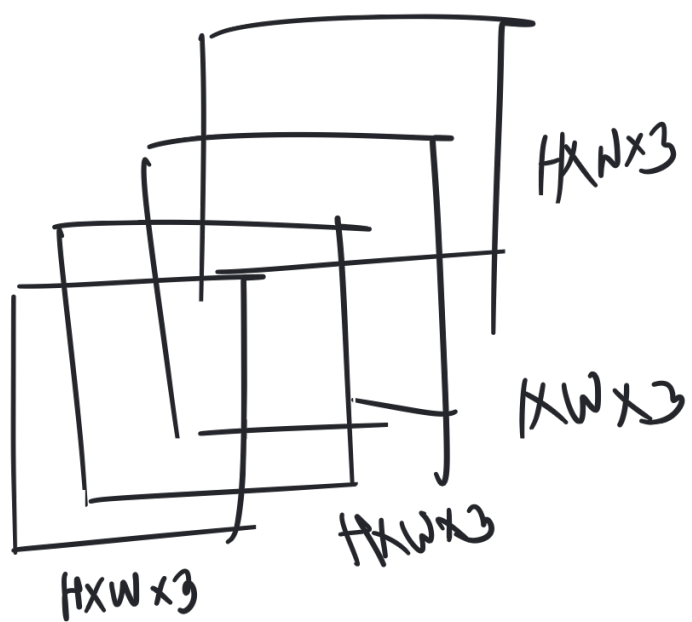
$$\text{Each } \underline{\text{patch}} = 1 \times 16 \times 16 \times 3$$



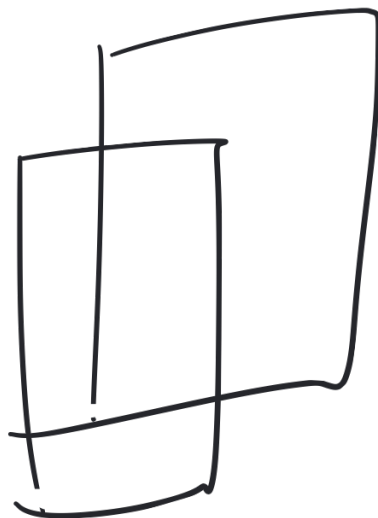
$H \times W \times 3 \rightarrow 6 \text{ Patches}$

$4 \times 6 \text{ Patches}$

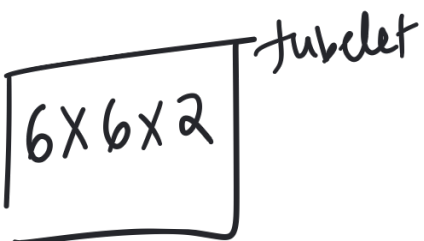
$4 \times H \times W \times 3$



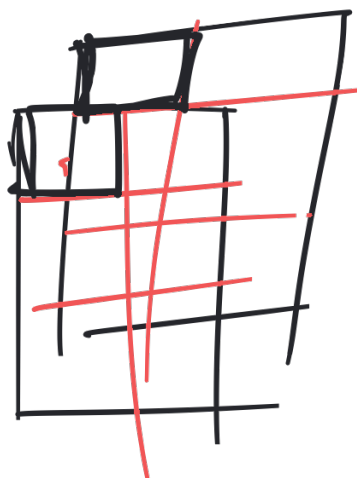
tubel



tubaldepth = 2

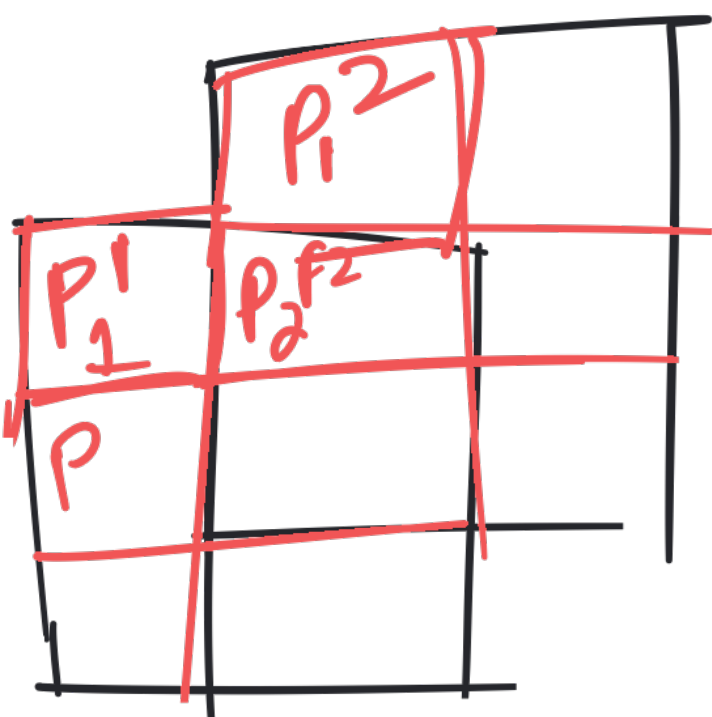


tubel

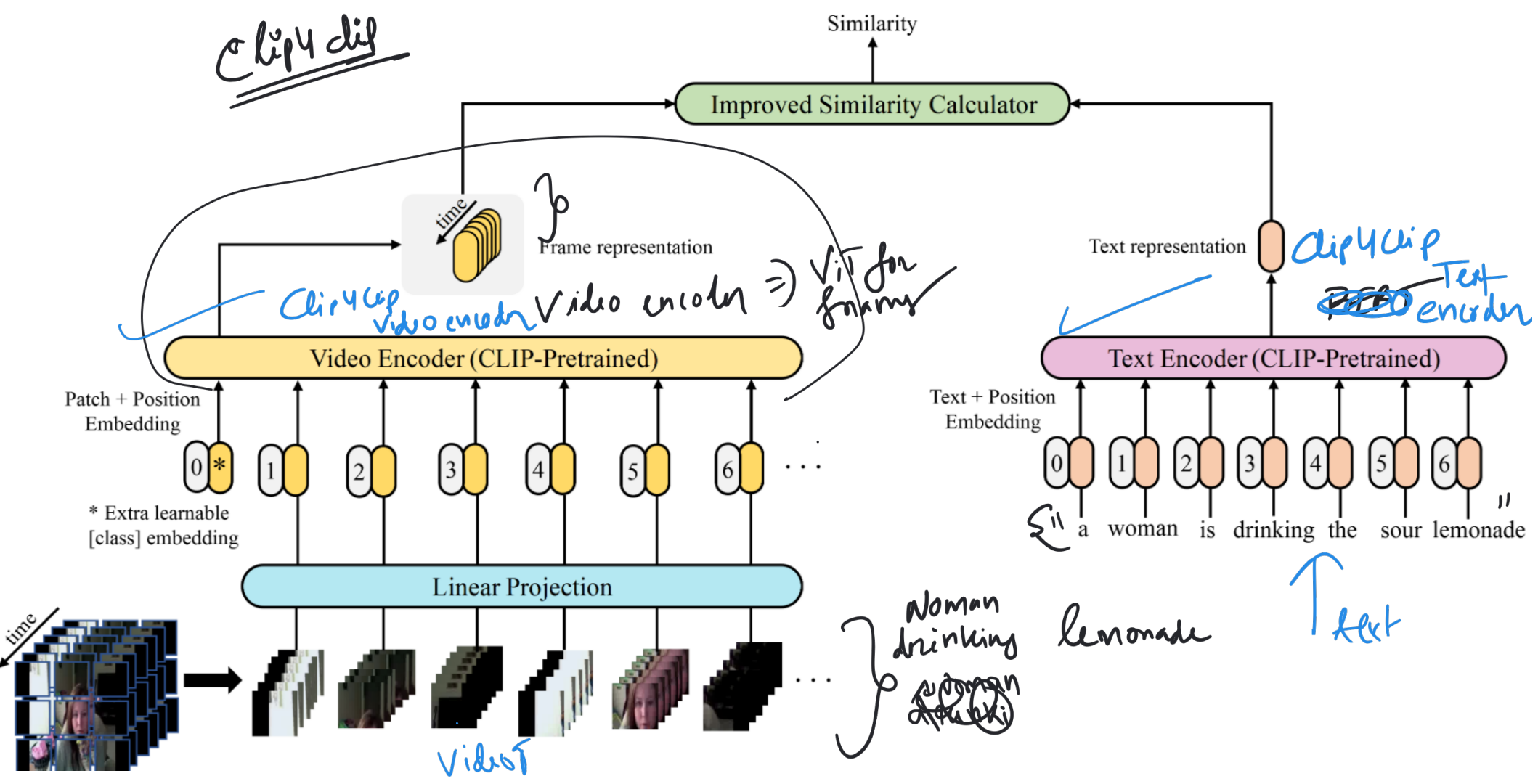


~~6x6x2~~ $P \times P \times 2$

$P \times P$

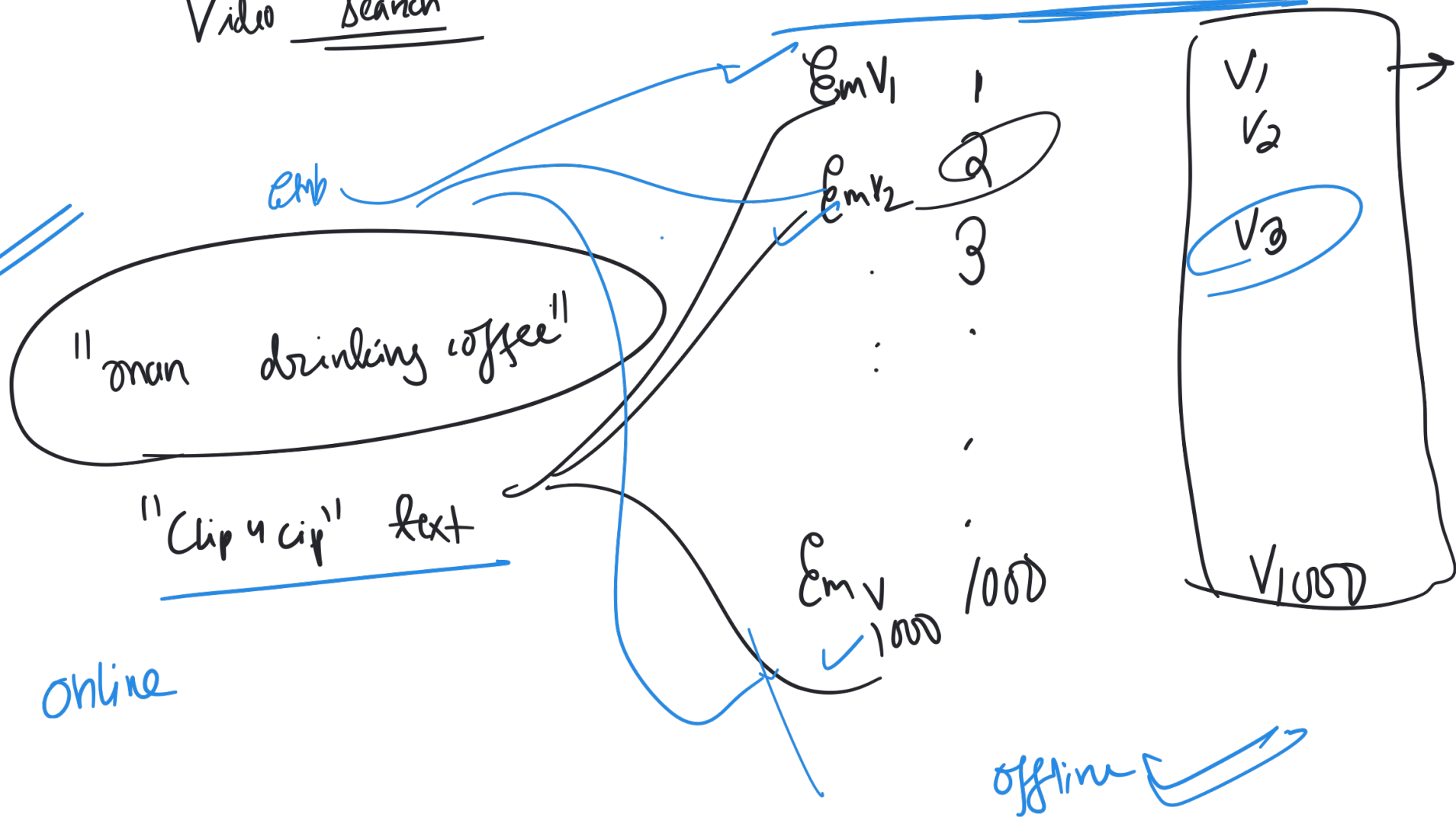


2 frame $\times 16 \times 16 \times 3$



Video search

Clip4Clip "Video Encoder"



Contrastive Loss

Anchor

+ve

-ve

Video embedder

3 × 10 × 10
frames patches

Video encoder

"woman
drinks
lemonade"

BERT

"man
fighting
dog"

BERT

Video of girl drinking Lemonade = $3 \times 160 \times 160 \times 3$

$3 \times 10 \times 10$ ~~patches~~ each Patch = $16 \times 16 \times 3$

Clip4Clip =
Given a video and text — convert both into vectors
(similarity)

Video = frames of imgs = a woman drinking lemonade
text = "A woman drinking the same lemonade"

